```
  #type SsNode[]
  children

  #type tuple(k)[]
  points

  #type boolean
  Leaf

  function SsNode(leaf, points=[], children=[])

class SsTree
```

```
  #type SsNode
  root
```

```
  #type integer
  m
  #type integer
  M
  #type integer
  k

  function SsTree(k, m, M)
```

Notice how, in figure 10.9, we represented our tree nodes like a list of “spheres”, each of which has a link to a child. We could, of course, add a type `SsSphere`, and keep a link to each sphere only children node as a field of this new type. It wouldn’t, though, make a great design, it would lead to data duplication (then both `SsNode` and `SsSphere` would hold fields for centroids and radius) and create an unnecessary level of indirection. Just keep in mind, when you look at the diagrams of SS-trees in these pages, that what is shown as components of a tree node are actually its children.